Projeto MayaRpg App

Requisitos da disciplina Modelagem de Software e Arquitetura de Sistemas

INTEGRANTES DO PROJETO e RA'S

André Ferreira da Silva	-	25027670
Vitor de Camargo Natário	-	25027184
Rogerio Rodrigues Vieira	-	25028005
Cauan Moreira da Silva Lima	-	25027645

São Paulo

2026

Sumário

1. Qualidade de Software 4
1.1. Modelo que qualidade de software (Diagrama/Design) 6
1.2. Processo (plano) de gerenciamento de qualidade de software (texto explicativo) 7
1.3. Identificação de atributos de qualidade da norma 25010. 9
1.4. Relatório que explica como a norma de qualidade de software 25010 é utilizada no processo de desenvolvimento. 10
2. Teste de Software 11
2.1. Plano de Teste 11
2.2. Apresentar 2 testes unitários. 11
2.3. Apresentar 2 testes de integração 11
2.4. Apresentar um teste de usuário (SISTEMA) 11
4. REFERÊNCIAS BIBLIOGRÁFICAS 12

INTRODUÇÃO

Contexto e Problema

A reabilitação fisioterápica exige consistência e acompanhamento preciso para garantir a eficácia do tratamento. No entanto, um dos maiores desafios clínicos é a baixa adesão dos pacientes, muitas vezes causada pela natureza repetitiva dos exercícios, e o risco de lesões decorrentes de execuções incorretas ou falhas no registro de progresso. O presente projeto surge para solucionar essa lacuna, propondo um **Software de Acompanhamento Fisioterápico Gamificado**, que transforma o processo de recuperação em uma experiência interativa e segura.

Proposta e Relevância

A proposta consiste em uma aplicação que une saúde e tecnologia para monitorar a evolução do paciente em tempo real. Academicamente, o projeto se destaca por aplicar o modelo **ISO/IEC 25010** e a cultura **DevOps** em um cenário de missão crítica. Na prática, sua relevância é vital: como falhas de sistema em softwares de saúde podem resultar em danos físicos diretos ao usuário, a **Confiabilidade** e a **Segurança** são os pilares centrais deste desenvolvimento.

Público-Alvo e Objetivos

O sistema foi projetado para um público diversificado, abrangendo desde **jovens até idosos**, o que exige uma interface intuitiva e adaptável. Os principais objetivos incluem:

- Aumentar o engajamento do paciente através de elementos de gamificação.
- Garantir a integridade total dos dados clínicos e do histórico de exercícios.
- Prover uma plataforma estável que suporte o acompanhamento remoto seguro.

Tecnologias Utilizadas

Para sustentar esses requisitos, o projeto utiliza um stack tecnológico robusto e escalável:

- **Backend:** Java com **Spring Boot**, utilizando **JPA/Hibernate** para persistência e **Spring Security** para proteção de dados sensíveis.
- **Banco de Dados:** Gerenciamento de migrações via **Flyway/Liquibase** para garantir consistência.
- **Frontend/Design:** Prototipagem de alta fidelidade no **Figma**.
- **Qualidade e Testes:** Automação com **JUnit**, testes de integração e gestão de defeitos via **Jira**.
- **Infraestrutura:** Containerização com **Docker**, garantindo a portabilidade entre ambientes de desenvolvimento e produção.

Escopo do Projeto

O escopo abrange o desenvolvimento do **MVP (Minimum Viable Product)**, focado no núcleo de gamificação, autenticação segura e persistência de dados. **Não faz parte da proposta**, neste momento, a integração direta com sensores de hardware externos (como wearables complexos) ou a implementação de teleconsultas via vídeo em tempo real, mantendo o foco na validação da lógica de exercícios e segurança da informação.

Abordagem de Qualidade

Este documento detalha como a qualidade é tratada não como uma etapa final, mas como um processo contínuo através do **Shift-Left Testing**. Desde a análise de requisitos até o deploy final, cada fase conta com validações específicas para assegurar que o software entregue não apenas funcionalidade, mas segurança física ao seu usuário final

1. Qualidade de Software

Neste tópico, vamos detalhar a **Definição de Qualidade** específica para o seu software. O foco aqui é transformar os conceitos da norma ISO/IEC 25010 em ações práticas dentro do seu fluxo de desenvolvimento (DevOps).

Definição e Essência da Qualidade no Projeto

Para este sistema, qualidade não é apenas a ausência de bugs, mas a **garantia de integridade física e clínica do usuário**. Dado que a aplicação lida com a saúde de pacientes de diversas faixas etárias, a qualidade é o alicerce que permite que a gamificação seja uma ferramenta terapêutica segura e não um risco adicional.

Fundamentação na ISO/IEC 25010

A qualidade do software será avaliada sob a ótica da norma **ISO/IEC 25010**, que categoriza as características intrínsecas do produto. No contexto deste projeto, a qualidade se manifesta em duas frentes:

1. **Requisitos Funcionais:** O sistema deve executar com precisão as regras de negócio da fisioterapia (contagem de repetições, níveis de dificuldade e biofeedback).
2. **Requisitos Não Funcionais:** Atributos que garantem a viabilidade e segurança da operação.

Atributos Críticos de Qualidade

Baseado na norma, definimos os seguintes pilares como essenciais:

- **Confiabilidade (Reliability):** Essencial para a persistência de dados. O uso de **JPA/Hibernate** garante que o progresso do paciente não seja perdido, evitando que o tratamento retroceda por falhas técnicas.

- **Segurança (Security):** Como lidamos com dados sensíveis de saúde, a implementação do **Spring Security** e auditorias constantes garantem o sigilo e a integridade das informações, protegendo o acesso indevido aos protocolos de exercícios.
- **Usabilidade (Usability):** A qualidade aqui é medida pela facilidade de uso tanto por jovens quanto por idosos. A validação prévia em **Figma** foca na ergonomia cognitiva, garantindo que o aspecto lúdico (gamificação) não oculte a funcionalidade clínica.
- **Desempenho e Eficiência:** O sistema deve responder em tempo real durante os exercícios para evitar frustração ou execução de movimentos em ritmo inadequado pelo paciente.
- **Manutenibilidade:** Através da containerização com **Docker** e revisões de código (**Code Review**), garantimos que o software possa evoluir e receber correções sem comprometer as funcionalidades existentes.

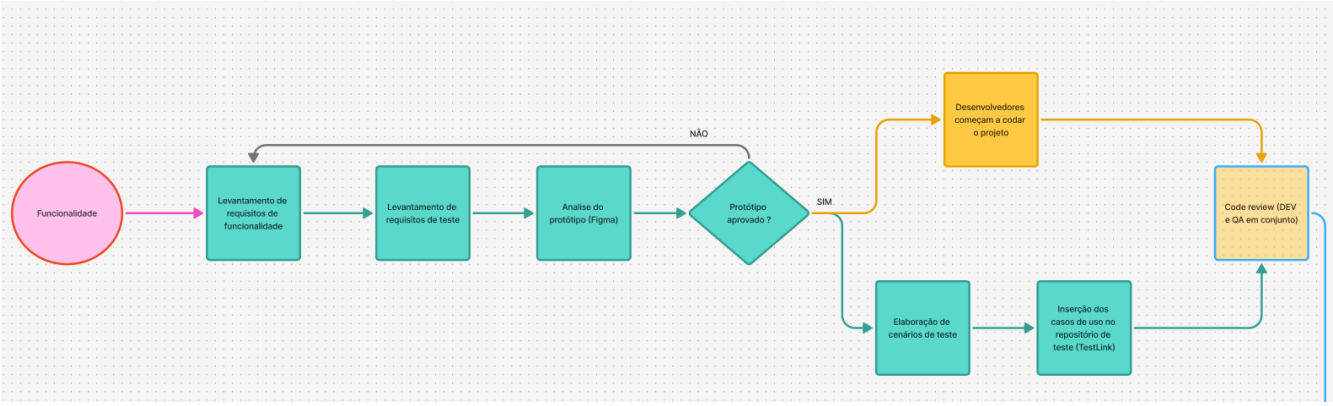
Medição e Garantia da Qualidade

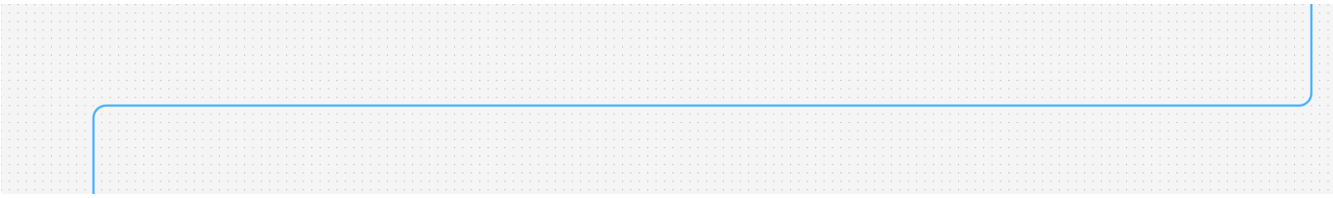
A equipe não apenas define a qualidade, mas a monitora através de métricas e processos rigorosos integrados ao ciclo DevOps:

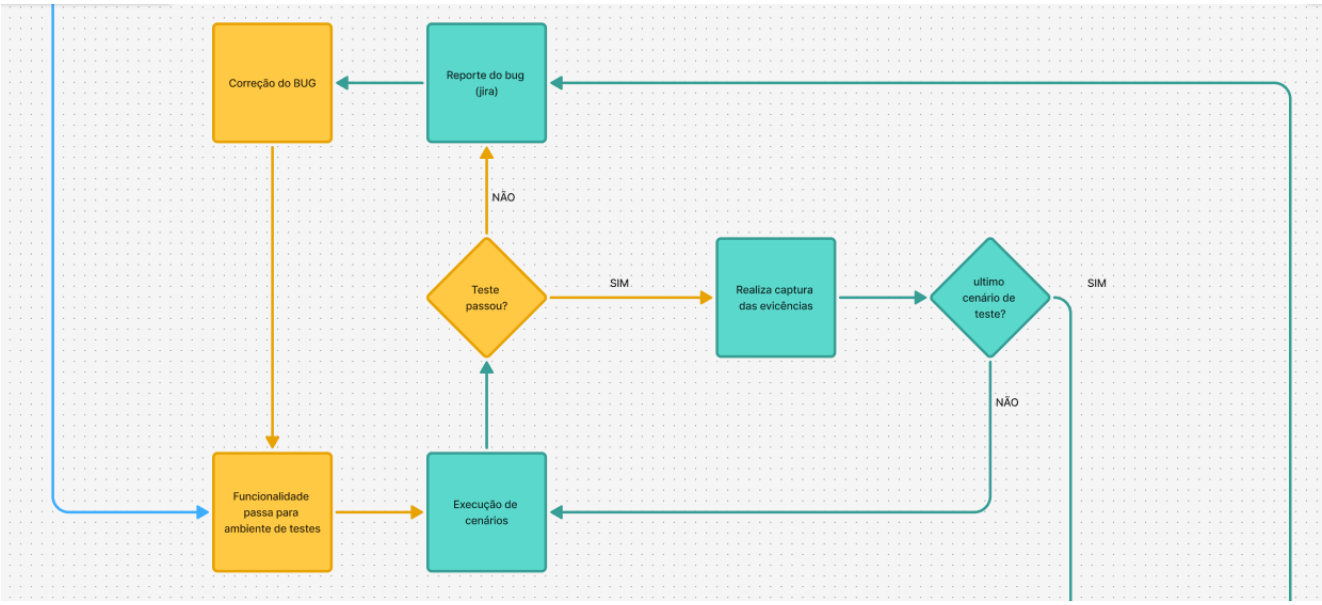
- **Shift-Left Testing:** A qualidade começa na análise de requisitos. Ao validar protótipos e critérios de aceite precocemente, reduzimos o custo de correção de erros de lógica.
- **Automação de Testes:** Utilização de **JUnit** para testes unitários e ferramentas de automação para garantir que cada incremento de código mantenha a estabilidade do sistema.
- **Gestão de Migrações:** O uso de **Flyway ou Liquibase** garante que a estrutura do banco de dados seja idêntica em todos os ambientes, eliminando o risco de erros de persistência durante o deploy.
- **Rastreabilidade:** Através do **Jira**, cada bug identificado é vinculado a um requisito, permitindo medir a densidade de defeitos e a eficácia das correções antes de cada release.

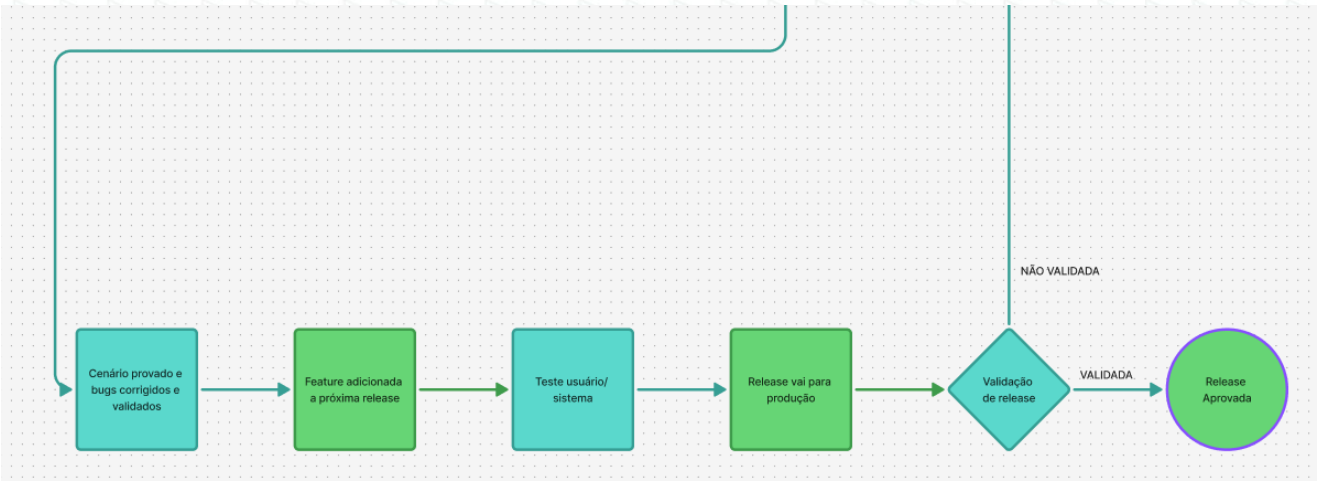
Dessa forma, a qualidade no Software de Acompanhamento Fisioterápico é uma responsabilidade compartilhada entre desenvolvedores e QA, garantindo que a entrega final seja robusta, segura e clinicamente eficaz.

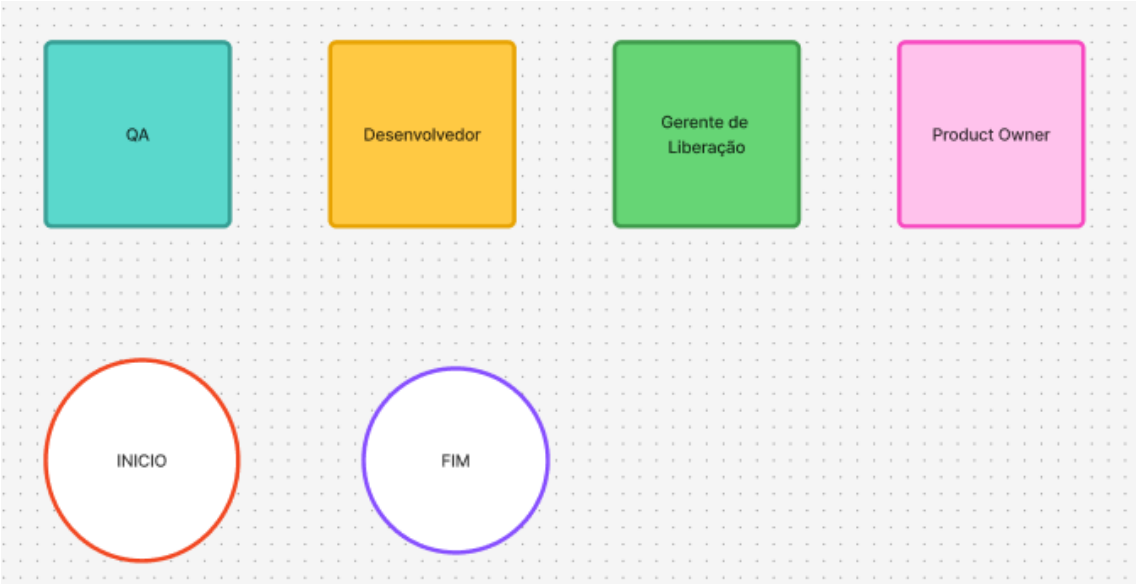
1.1. Modelo que qualidade de software (Diagrama/Design)











1.2. Processo (plano) de gerenciamento de qualidade de software (texto explicativo)

Gerenciamento da qualidade neste projeto é estruturado em três frentes: **Planejamento** (definição de padrões), **Controle** (verificação técnica) e **Monitoramento** (acompanhamento contínuo), garantindo que o software evolua de forma segura e sustentável.

Práticas de Desenvolvimento e Versionamento

A base da nossa qualidade reside no controle rigoroso do código-fonte e do ambiente:

- **Versionamento (Git):** Utilizaremos o **Git** com uma estratégia de ramificação (como *Gitflow*), onde o código em produção está sempre estável. Nenhuma alteração é mesclada à ramificação principal sem passar por validação.
- **Revisão de Código (Code Review):** Implementamos a revisão conjunta entre desenvolvedores (DEV) e analistas de qualidade (QA). O objetivo é garantir que o código siga padrões de limpeza, segurança e, acima de tudo, testabilidade.
- **Containerização (Docker):** Para garantir que o software funcione de forma idêntica em "minha máquina", homologação e produção, utilizamos o Docker, eliminando erros de ambiente que prejudicam a confiabilidade.

Estratégia de Testes e Validação

A verificação da qualidade ocorre em múltiplas camadas:

- **Testes Automatizados:** Implementamos testes unitários e de integração com **JUnit**. Estes testes são executados automaticamente a cada alteração, garantindo que novas funcionalidades não quebrem o que já está funcionando (testes de regressão).
- **Gestão de Banco de Dados:** O uso de **Flyway/Liquibase** permite o controle de versão do banco de dados, garantindo que a persistência (ponto crítico do projeto) seja consistente em todos os deploys.
- **Validação com Usuário:** Antes do deploy final, realizamos testes de usabilidade baseados nos protótipos do **Figma** e sessões de teste com o usuário final (Beta Testing), garantindo que a gamificação seja intuitiva para todas as idades.

Responsabilidades e Critérios de Aceite

A qualidade é uma responsabilidade compartilhada, mas com papéis definidos:

- **Desenvolvedores:** Responsáveis pela criação de testes unitários e pela correção imediata de bugs críticos.
- **QA/Analista de Qualidade:** Responsável por desenhar os cenários de teste, validar os critérios de aceite e gerenciar a fila de defeitos no **Jira**.
- **Critérios de Aceite:** Cada funcionalidade só é considerada "Pronta" (*Definition of Done*) se passar por três filtros: cobertura de testes automatizados, revisão de código por um par e validação funcional em ambiente de homologação.

Gestão de Erros e Melhoria Contínua (Não Conformidades)

Quando uma não conformidade é detectada:

1. **Registro:** O erro é documentado no **Jira** com evidências, prioridade (baseada no risco físico ao paciente) e passos para reprodução.

2. **Análise de Causa Raiz:** Para erros críticos, a equipe realiza uma breve retrospectiva para entender por que o teste automatizado não pegou a falha, ajustando o processo para o futuro.
3. **Ciclo DevOps:** Feedbacks rápidos são coletados em cada milestone (como os definidos em 06/03 e 20/03), permitindo ajustes de rota imediatos e garantindo a melhoria contínua do produto final.

1.3. Identificação de atributos de qualidade da norma 25010.

Atributo (ISO 25010)	Aplicação no Projeto	Justificativa DevOps/Qualidade
Adequação Funcional	Testes com JUnit e ferramentas de automação.	Garante que as funções fisioterápicas cumpram o objetivo clínico.
Confiabilidade	Persistência de dados com JPA/Hibernate.	Evita perda de progresso do paciente e garante a integridade dos dados.
Segurança	Implementação de Spring Boot Security e Auditoria.	Proteção de dados sensíveis de saúde e controle de acesso aos exercícios.
Usabilidade	Análise de protótipo (Figma) e gamificação.	Essencial para a adesão de diferentes faixas etárias ao tratamento.
Portabilidade	Uso de Docker/Containerização .	Garante que o software funcione identicamente em dev, homologação e produção.

1.4. Relatório que explica como a norma de qualidade de software 25010 é utilizada no processo de desenvolvimento.

Tradução da Norma em Requisitos Não Funcionais (RNF)

A norma foi utilizada como o "dicionário de qualidade" na fase de **Definição de Requisitos (27/02)**. Cada atributo da ISO 25010 foi desdobrado em um requisito técnico mensurável:

- **Adequação Funcional:** Definida pela completude das rotinas de fisioterapia. Se o sistema não cobre a amplitude de movimento necessária, ele falha na sua finalidade clínica.
- **Confiabilidade:** Traduzida na obrigatoriedade de **Database Migrations**. A métrica de sucesso é a taxa de erro zero em transações de banco de dados durante quedas simuladas de conexão.
- **Segurança:** Implementada através de criptografia e controle de acesso (Spring Security), protegendo a integridade dos dados de saúde contra modificações não autorizadas.
- **Manutenibilidade:** Definida pela modularidade do código. Aqui, estabelecemos que o código deve ser facilmente testável e documentado.

Verificação de Qualidade via Métricas (JaCoCo e HTML Coverage)

Para garantir que a norma fosse aplicada e não apenas citada, adotamos o **JaCoCo (Java Code Coverage)** como nossa principal ferramenta de auditoria de qualidade.

- **Critério de Aceite:** Estabelecemos um *threshold* (limite mínimo) de cobertura de código. Não basta o código funcionar; ele deve ser testado em seus diversos caminhos lógicos.
- **Uso do HTML Coverage:** Os relatórios gerados pelo JaCoCo em formato HTML são utilizados em cada **Code Review**. Eles nos permitem visualizar exatamente quais linhas de lógica de negócio (especialmente as críticas de cálculo fisioterápico) estão protegidas por testes unitários e quais estão "descobertas".
- **Melhoria Contínua:** Se um relatório do JaCoCo aponta baixa cobertura em um módulo de persistência, isso aciona automaticamente uma tarefa de refatoração para atingir o padrão de **Confiabilidade** exigido pela norma.

Integração ao Ciclo DevOps (Shift-Left)

A aplicação da norma ocorre de forma antecipada no fluxo:

1. **Design (Figma):** A **Usabilidade** (ISO 25010) é validada no protótipo antes de qualquer linha de código, reduzindo o retrabalho.
2. **Desenvolvimento:** O desenvolvedor escreve o código já visando as métricas do JaCoCo.

3. **Smoke Tests (06/03):** Validam a **Portabilidade** e **Operacionalidade** básica em containers Docker.

4. **Teste de Regressão:** Garante que a **Manutenibilidade** está sendo respeitada, verificando se novas funções não degradam atributos de qualidade já estabelecidos.

Conclusão da Aplicabilidade

A ISO/IEC 25010 serviu como o "norte" para nossas decisões de arquitetura. Ao unir a norma com ferramentas de medição objetiva, como o **JaCoCo**, transformamos conceitos subjetivos de "qualidade" em dados binários: ou o código atende aos critérios de cobertura e segurança estipulados, ou ele não avança para o deploy final. Essa rigorosidade é o que garante a segurança física do paciente, objetivo maior deste projeto.

2. Teste de Software

2.1. Plano de Teste

2.2. Apresentar 2 testes unitários.

2.3. Apresentar 2 testes de integração

2.4. Apresentar um teste de usuário (SISTEMA)

4. REFERÊNCIAS BIBLIOGRÁFICAS

Normas e Fundamentos de Engenharia

- **AISO/IEC 25010:2023**: https://iso25000.com/index.php/en/iso-25000-standards/iso-25010

2. Automação, DevOps e Qualidade (Livros em Português)

- **DevOps na Prática (Casa do Código):** https://www.casadocodigo.com.br/products/livro-devops

3. Documentação das Ferramentas (Essencial para o Relatório)

- **JaCoCo (Java Code Coverage) - Documentação Oficial:** https://www.jacoco.org/jacoco/trunk/doc/
- **JUnit 5 (User Guide):** https://junit.org/junit5/docs/current/user-guide/
- **Spring Boot Security (Reference Guide):** https://docs.spring.io/spring-security/reference/index.html
- **Flyway (Database Migrations):** https://documentation.red-gate.com/fd/flyway-documentation-138346303.html